

Phần 4: Chiến Lược Tham Lam – Chiến Lược Nhánh Và Cận

Bài 1:

Cho mảng A có n phần tử. Hãy bố trí lại thứ tự các phần tử trong mảng A sao tổng lớn nhất của $A[i]*i$ là lớn nhất. Cài đặt và giải thích việc áp dụng chiến lược tham lam vào bài toán.

- Ý tưởng/Giải thích:

+ Muốn $A[i]*i$ là lớn nhất thì chỉ được lấy $A[i]$ lớn nhất nhân với số i lớn nhất

=> Sắp xếp lại mảng tăng dần để $A[i]$ lớn nhất ở cuối mảng

+ Cho dù $A[i]$ có số âm thì số âm vẫn sẽ ở đầu mảng nên số index i vẫn sẽ là nhỏ nhất để khi cộng lại vẫn sẽ ra tổng lớn nhất

- Code:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int n;
    cout << "Nhap n: ";
    cin >> n;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin >> a[i];
    sort(a.begin(), a.end());

    long long res = 0;
    for(int i = 0; i < n; i++){
        res += 1LL * a[i] * i;
    }
    cout << "Ket qua: " << res;

    return 0;
}
```

- Ví dụ:

Input: $n = 6$

$a = \{8, 2, -5, 1, 0, 3\}$

$\Rightarrow \text{sort } a \Rightarrow a = \{-5, 0, 1, 2, 3, 8\}$

$\Rightarrow \text{Nhân } a[i] \text{ với } i \Rightarrow \text{res} = (-5) * 0 + 0 * 1 + 1 * 2 + 2 * 3 + 3 * 4 + 8 * 5 = 60$

$\Rightarrow \text{res} = 60$

Output: 60

- Kết quả chạy code:

```
Nhap n: 6
8
2
-5
1
0
3
Ket qua: 60
```

Bài 2:

Cho mảng A gồm các số từ 0 đến 9. Hãy tìm tổng nhỏ nhất của 2 số được tạo từ các số trong mảng A (khi tạo số cần sử dụng tất cả các phần tử trong mảng A).

Ví dụ: Ta có mảng $A = \{2, 1, 4, 3, 5\}$ thì 2 số nhỏ nhất được tạo từ các phần tử trong mảng A là 24 và 135 nên tổng sẽ là 159.

- **Ý tưởng/Giải thích:** Giá trị của số phụ thuộc vào nhiều vào chữ số bên trái hơn là số ở bên phải (ở hàng đơn vị). Ví dụ: số 1xx2 nhỏ hơn 2xx1 rất nhiều

=> Bước 1: Sort tăng dần → các số nhỏ đứng trước để sắp xếp trước

=> Bước 2: Phân phối xen kẽ giữa 2 số n1 và n2. Lần lượt gán các số từ mảng a vào:

- vị trí chẵn → số n1
- vị trí lẻ → số n2

=> Điều này sẽ giúp cho:

- 2 số có độ dài gần bằng nhau (không thể dồn nhiều số nhỏ vào 1 số vì sẽ làm số đó ngày càng lớn hơn. Ví dụ: 1111 và 99 thay vì là 119 và 119)
- chữ số nhỏ nằm ở hàng cao và chữ số lớn nằm ở hàng đơn vị → Làm tổng ở các hàng cao nhỏ nhất có thể

- **Code:**

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main(){
    int n;
    cout << "Nhap n: ";
    cin >> n;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin >> a[i];

    sort(a.begin(), a.end());
    long long n1 = 0, n2 = 0;
    for(int i = 0; i < a.size(); i++){
        if(i % 2 == 0) n1 = n1 * 10 + a[i];
        else n2 = n2 * 10 + a[i];
    }
    cout << "Ket qua: " << n1 + n2;
    return 0;
}
```

- Ví dụ:

Input: $n = 5$

$a = \{2, 1, 4, 3, 5\};$

=> Bước 1: Sort $a \rightarrow a = \{1, 2, 3, 4, 5\}$ và khai báo $n1 = 0, n2 = 0$

=> Bước 2: Xét từ $a[0] = 1$:

$$n1 = 0 * 10 + 1 = 1$$

$$n2 = 0$$

=> Bước 3: Xét từ $a[1] = 2$:

$$n1 = 1$$

$$n2 = 0 * 10 + 2 = 2$$

=> Bước 4: Xét từ $a[2] = 3$:

$$n1 = 1 * 10 + 3 = 13$$

$$n2 = 2$$

=> Bước 5: Xét từ $a[3] = 4$:

$$n1 = 13$$

$$n2 = 2 * 10 + 4 = 24$$

=> Bước 6: Xét từ $a[4] = 5$:

$$n1 = 13 * 10 + 5 = 135$$

$$n2 = 24$$

=> Bước 7: Tổng $n1 + n2 = 135 + 24 = 159$

=> Output: 159

+ Ví dụ khi xét các số khác như: $n1 = 12345$ và $n2 = 0 \Rightarrow$ Tổng $= 12345 > 159$. Hay $n1 = 123$ và $n2 = 45 \Rightarrow$ Tổng $= 167 > 159$. Hay $n1 = 134$ và $n2 = 25 \Rightarrow$ Tổng $159 = 159$ (giống với output). Như vậy, trường hợp tối ưu nhất để ra được tổng nhỏ nhất là luôn để các số nhỏ ở hàng cao và các số lớn ở hàng đơn vị

- Kết quả chạy code:

```
Nhap n: 5
2
1
4
3
5
Ket qua: 159
```

Bài 3:

Sử dụng giải thuật tham lam cho bài toán được mô tả như sau:

Một người cần mua gạo cho gia đình tại một cửa hàng gạo(cửa hàng đóng cửa vào ngày chủ nhật).

Bài toán có các tham số sau:

- A là số lượng gạo người đó có thể mua nhiều nhất trong 1 ngày**
- B là số lượng ngày mà gia đình người đó cần gạo để dùng.**
- C là số lượng gạo gia đình mỗi ngày gia đình đó cần.**

Giả sử đang ở ngày thứ 2 trong tuần và cần gạo cho B ngày sắp tới.

Yêu cầu bài toán cần xác định số lượng ngày ít nhất người đó cần phải mua gạo để đáp ứng đủ cho nhu cầu gia đình.

- Ý tưởng/Giải thích:

- + Không mua được ngày CN → 1 tuần chỉ mua 6 ngày → Mua được $A * 6$ trong 1 tuần
- + Mỗi ngày đều tiêu thụ 1 lượng C gạo → Ăn hết $C * 7$ trong 1 tuần
- + Cần qua B ngày → Tổng gạo cần để dùng: $B * C$
- + Số ngày mua ít nhất sẽ là $(B * C) / A$

Với: Điều kiện là $A * 6 \geq C * 7$ nếu $A * 6 < C * 7$ tức là không đủ gạo $\rightarrow$ return -1

- Code:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int A, B, C;
    cout << "Nhap A, B, C: ";
    cin >> A >> B >> C;
    if(A * 6 < C * 7) return -1;
    cout << "So luong ngay: " << ceil((double)(B * C) / A);
    return 0;
}
```

- Ví dụ:

Input: A = 10, B = 7, C = 2

$\Rightarrow$ Check $A * 6 \geq C * 7 \rightarrow 10 * 6 \geq 2 * 7 \rightarrow 60 \geq 14$ (đúng)

$\Rightarrow$ Tính kết quả $(B * C) / A = (7 * 2) / 10 = 1.4$

$\Rightarrow$ Làm tròn lên (ceil) $\rightarrow \text{ceil}(1.4) = 2$

$\Rightarrow$ Output = 2

+ Giả sử trường hợp không được:

Input: A = 8, B = 8, C = 7

$\Rightarrow$ Check $A * 6 \geq C * 7 \rightarrow 8 * 6 \geq 7 * 7 \rightarrow 48 \geq 49$ (sai)

- Kết quả chạy code:

```
Nhap A, B, C: 10
7
2
So luong ngay: 2
```

Bài 4:

Cho chuỗi ký tự S gồm các ký tự in hoa. Sử dụng chiến lược tham lam để kiểm tra xem có thể sắp xếp lại các ký tự trong chuỗi S để 2 ký tự liên kề nhau không giống nhau hay không.

Ví dụ:

- Chuỗi S = AAABBC thì có thể sắp lại chuỗi S theo yêu cầu.

- Chuỗi S = AAAABB thì không thể sắp lại chuỗi S theo yêu cầu.

- Ý tưởng/Giải thích:

+ Nếu muốn sắp xếp 1 chuỗi xen kẽ từ AAABBC sang ABABAC thì cứ cách mỗi ký tự, chèn vào một ký tự khác với ký tự trước → Nếu không thể chèn ký tự khác tức là số lượng ký tự trùng nhau chiếm phần lớn trong chuỗi

+ Điều kiện: Ký tự xuất hiện nhiều nhất $\leq (n+1) / 2$ với $n = S.length()$. Nếu vượt quá $(n+1) / 2 \rightarrow$ không thể tạo các chuỗi xen kẽ

- Code:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    string s;
    cout << "Nhap chuoi: ";
    cin >> s;
```

```

unordered_map<char, int> mp;
for(char c : s) mp[c]++;

int maxFreq = INT_MIN;
for(auto &[key, freq] : mp){
    maxFreq = max(maxFreq, freq);
}

if(maxFreq <= (s.length() + 1) / 2) cout << "Co the sap xep chuoi S" << endl;
else cout << "Khong the sap xep chuoi S" << endl;
return 0;
}

```

- Ví dụ:

Input: s = AAABBC

=> Ta có map:

A : 3

B : 2

C : 1

=> Có maxFreq = 3 (từ key A)

=> Check maxFreq <= (s.length() + 1) / 2 → 3 <= (6 + 1) / 2 → 3 <= 3 (đúng)

=> Sắp xếp được (Output)

+ Giả sử trường hợp không được:

Input: s = AAAABB

=> Ta có map:

A : 4

B : 2

=> Có maxFreq = 4 (từ key A)

=> Check maxFreq <= (s.length() + 1) / 2 → 4 <= (6 + 1) / 2 → 4 <= 3 (sai)

=> Không sắp xếp được (Output)

- Kết quả chạy code:

+ Sắp xếp được:

```
Nhap chuoì: AAABBC
Co the sap xep chuoì S
```

+ Không sắp xếp được:

```
Nhap chuoì: AAAABB
Khong the sap xep chuoì S
```

Bài 5:

Mô tả và cài đặt bài toán đổi tiền sử dụng chiến lược tham lam và chiến lược quy hoạch động. So sánh 2 phương pháp.

- Ý tưởng/Giải thích:

+ Greedy: chọn đồng lớn nhất rồi trừ bớt tiền. Cách này sẽ có thể ra kết quả không đúng

+ DP: xét tất cả khả năng lấy để có được đúng số tiền đó rồi tính max ở số tiền cuối

- Code:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Greedy
```

```

int greedy(vector<int> c, int S){
    sort(c.rbegin(), c.rend());
    int cnt = 0;
    for(int coin : c){
        cnt += S / coin;
        S %= coin;
    }
    return (S == 0) ? cnt : -1;
}

```

// DP

```

int dp(vector<int>& c, int S){
    vector<int> dp(S+1, INT_MAX);
    dp[0] = 0;
    for(int i = 1; i <= S; i++){
        for(int coin : c){
            if(i >= coin && dp[i-coin] != INT_MAX)
                dp[i] = min(dp[i], dp[i-coin] + 1);
        }
    }
    if(dp[S] == INT_MAX) return -1;
    return dp[S];
}

```

```

int main(){
    vector<int> c = {3, 5, 7};

```

```

cout << "Greedy: " << greedy(c, 13) << endl;
cout << "DP: " << dp(c, 13);
}

```

- **Ví dụ:** Cho coin là {3, 5, 7} và target S = 13

Greedy:

=> Luôn lấy xu lớn nhất => S = 13 nên lấy được xu 7

=> $cnt += 13/7 = 1 \Rightarrow cnt = 1$

=> $S = 13 \% 7 = 6$

=> Lấy tiếp xu 7 thì không được (vì $7 > 6$)

=> Lấy xu lớn tiếp theo là xu 5

=> $cnt = 2$ và $S = 1$

=> Lấy xu 5 không được xong qua xu 3 cũng không lấy được

=> $cnt = 2$ và $S = 1$, check ($S == 0$) ? $cnt : -1$; vì $S = 1$ nên return -1;

=> Output: -1

DP:

Mảng dp ban đầu: (gọi $M = INT_MAX$)

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13
i	0	M	M	M	M	M	M	M	M	M	M	M	M	M

=> Vì cần 0 tiền nên sẽ cần 0 đồng xu => $dp[0] = 0$

=> Check $i \geq coin \ \&\& \ dp[i-coin] \neq INT_MAX$ có $i \geq coin$ là false → skip

=> Tương tự với $i = 2 \rightarrow$ skip đến $i = 3$

=> Check $i \geq coin \ \&\& \ dp[i-coin] \neq INT_MAX \rightarrow true \rightarrow dp[3] = \min(dp[i], dp[i-coin] + 1) = \min(max, 0 + 1) = 1$. Tức là ở $dp[3]$ thì sẽ có 1 đồng 3 xu

Mảng dp mới:

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13
i	0	M	M	1	M	M	M	M	M	M	M	M	M	M

=> Check tiếp cho 4 thấy $dp[i-coin] \neq INT_MAX$ tức $dp[1] = \max \rightarrow \text{false} \rightarrow \text{skip}$

=> Với $i = 5$ thì $dp[i-coin] \neq INT_MAX$ có $dp[5-3] = dp[2] \rightarrow \text{false}$, check tiếp $coin = 5$
 $\rightarrow dp[0] \neq \max \rightarrow \text{true} \rightarrow dp[5] = \min(dp[i], dp[i-coin] + 1) = \min(\max, 0 + 1) = 1$

Mảng dp mới:

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13
i	0	M	M	1	M	1	M	M	M	M	M	M	M	M

=> Với $i = 6$, $dp[6-3] \neq \max \rightarrow dp[6] = \min(dp[6], dp[3] + 1) = \min(\max, 1 + 1) = 1 + 1$
 $= 2$. Với $coin = 5$ thì $dp[6-5] == \max \rightarrow \text{false}$

Mảng dp mới:

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13
i	0	M	M	1	M	1	2	M	M	M	M	M	M	M

=> Tương tự với i chạy từ 7 đến 13

Mảng dp hoàn chỉnh:

dp	0	1	2	3	4	5	6	7	8	9	10	11	12	13
i	0	M	M	1	M	1	2	1	2	3	2	3	2	3

=> Output: 3

Chỉnh lại coin cho Greedy:

Nếu muốn Greedy có kết quả thì có thể thêm 1 xu vào coin => $coin = \{1, 3, 5, 7\}$ thì khi chạy $S = 13$ sẽ lấy 1 xu 7 đồng, 1 xu 5 đồng và 1 xu 1 đồng => Kết quả sẽ ra bằng 3

- Kết quả chạy code:

```
Greedy: -1
DP: 3
```

Bài 6:

Mô tả và cài đặt bài toán xếp ba lô (Knapsack) sử dụng chiến lược tham lam, chiến lược quy hoạch động và chiến lược quay lui nhánh cận. So sánh các phương pháp.

- Ý tưởng/Giải thích:

+ Greedy: chọn theo giá trị/khối lượng

- Là bài toán Fractional Knapsack
- Tính đơn giá (value/weight) cho từng đồ vật xong sắp xếp các đồ vật theo thứ tự đơn giá giảm dần
- Duyệt từng đồ vật theo thứ tự này:
 - Nếu còn đủ chỗ trong ba lô → lấy toàn bộ đồ vật.
 - Nếu không đủ chỗ → lấy một phần của đồ vật sao cho đầy ba lô.
- Dừng khi ba lô đầy hoặc hết đồ vật.

+ DP: tối ưu tuyệt đối vì xét toàn bộ các trường hợp

- Là bài toán 0/1 Knapsack (không chia nhỏ vật)
- Giá trị lớn nhất khi xét i vật đầu tiên với sức chứa w
- Với mỗi vật có 2 lựa chọn:
 - Không chọn vật i
 - Chọn vật i (nếu còn đủ sức chứa)

+ Quay lui nhánh cận: duyệt + cắt tỉa

- Giống việc giải quyết các bài toán backtracking
- Với mỗi vật có 2 nhánh:
 - Chọn
 - Không chọn
- Trong quá trình duyệt:
 - Tính giá trị hiện tại
 - Cập nhật kết quả tốt nhất (best)
- Nhánh cận (cắt tỉa): Nếu giá trị tối đa có thể đạt được trong tương lai không vượt best → bỏ luôn toàn bộ nhánh đó mà không cần xét tiếp

- Code:

```
#include <bits/stdc++.h>
```

```

using namespace std;

//greedy
struct Item {
    int wt, val;
};

bool cmp(Item a, Item b){
    return (double)a.val / a.wt > (double)b.val / b.wt;
}

double greedyKnapsack(int W, vector<Item> a){
    sort(a.begin(), a.end(), cmp);
    double total = 0;

    for(auto x : a){
        if(W >= x.wt){
            total += x.val;
            W -= x.wt;
        } else {
            total += x.val * ((double)W / x.wt);
            break;
        }
    }
    return total;
}

// dp
int knapsackDP(int W, vector<int>& wt, vector<int>& val){
    int n = wt.size();
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));

    for(int i = 1; i <= n; i++){
        for(int w = 0; w <= W; w++){
            if(wt[i-1] <= w){
                dp[i][w] = max(dp[i-1][w],
                               val[i-1] + dp[i-1][w - wt[i-1]]);
            }
        }
    }
    return dp[n][W];
}

```

```

        } else {
            dp[i][w] = dp[i-1][w];
        }
    }
}
return dp[n][W];
}

```

```

// backtracking
int best = 0;

```

```

void Try(int i, int curW, int curVal, int W,
        vector<int>& wt, vector<int>& val){
    if(i == wt.size()){
        best = max(best, curVal);
        return;
    }
}

```

```

// Không chọn

```

```

Try(i+1, curW, curVal, W, wt, val);

```

```

// Chọn

```

```

if(curW + wt[i] <= W){
    Try(i+1, curW + wt[i], curVal + val[i], W, wt, val);
}
}

```

```

int main(){
    int W = 50;
    vector<int> wt = {10,20,30};
    vector<int> val = {60,100,120};

    vector<Item> items;
    for(int i = 0; i < wt.size(); i++){
        items.push_back({wt[i], val[i]});
    }
}

```

```

cout << "Greedy (fractional): " << greedyKnapsack(W, items) << endl;
cout << "DP (Bai toan 0/1): " << knapsackDP(W, wt, val) << endl;
best = 0;
Try(0, 0, 0, W, wt, val);
cout << "Backtracking: " << best << endl;

return 0;
}

```

- Ví dụ:

Input:

+ Weight (Cân nặng): `vector<int> wt = {10,20,30};`

+ Value (Giá trị): `vector<int> val = {60,100,120};`

=> Items = `{[10 : 60], [20 : 100], [30 : 120]}`; (items là vector với pair là `wt[i]`, `val[i]`)

+ Công thức:

=> Greedy:

- Sort mảng đồ vật (items)
- Công thức: đơn giá = weight / value
- Cộng lại các giá trị sau mỗi lần lấy

=> DP:

- Chạy hết mảng dp để tìm ra vật các bỏ các đồ vật
- Công thức: $dp[i][w] = \max(dp[i-1][w], val[i] + dp[i-1][w-wt[i]])$ nếu $wt[i-1] \leq weight$. Không thì $dp[i][w] = dp[i-1][w]$. Tức là với mỗi vật có 2 lựa chọn:
 - Không chọn vật i
 - Chọn vật i (nếu còn đủ sức chứa)
- Trả về `dp[n][W]` (phần tử ở cuối mảng dp)

=> Backtracking: Xét các phần nhỏ của Try

- Thử Try 0 ban đầu với `Try(0, 0, 0, W, wt, val);`
- Check best sau mỗi lần đệ quy:

```

if(i == wt.size()){
    best = max(best, curVal);
}

```



```
    return;  
}
```

- Nếu không chọn: Try(i+1, curW + wt[i], curVal + val[i], W, wt, val)
- Nếu chọn: Try(i+1, curW, curVal, W, wt, val)

=> Output:

Greedy: 240

DP: 220

Backtracking: 220

- **Kết quả chạy code:**

```
Greedy (fractional): 240  
DP (Bai toan 0/1): 220  
Backtracking: 220
```

Bài 7:

Mô tả và cài đặt bài toán người du lịch (TSP) sử dụng chiến lược tham lam và chiến lược quay lui nhánh và cận. So sánh 2 phương pháp.

- **Ý tưởng/Giải thích:**

+ Greedy: chọn thành phố gần nhất

+ Quay lui và hánh cận: thử tất cả + cắt tỉa bớt các đường đi quá tốn kém

- **Code:** (Note: Cho sẵn đường đi ở trong code để khỏi phải nhập input nhiều lần)

```
#include <bits/stdc++.h>  
using namespace std;
```

```
int n = 4;  
int cost[4][4] = {
```

```

        {0,10,15,20},
        {10,0,35,25},
        {15,35,0,30},
        {20,25,30,0}
};

int visited[20], best = INT_MAX;

void Try(int u, int cnt, int sum){
    if(cnt == n){
        best = min(best, sum + cost[u][0]);
        return;
    }
    for(int v=0;v<n;v++){
        if(!visited[v]){
            visited[v]=1;
            Try(v, cnt+1, sum+cost[u][v]);
            visited[v]=0;
        }
    }
}

int main(){
    visited[0]=1;
    Try(0,1,0);
    cout << best;
}

```

- Ví dụ:

Input: n = 4;

```

cost[4][4] = {
    {0,10,15,20},
    {10,0,35,25},
    {15,35,0,30},
    {20,25,30,0}
};

```

=> Ma trận thành phố:

Index	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

=> Từ thành phố hiện tại → chọn thành phố gần nhất chưa đi

Greedy:

=> Bắt đầu từ 0:

- Từ 0 → chọn nhỏ nhất: 1 (10)
- Từ 1 → chọn: 3 (25)
- Từ 3 → chọn: 2 (30)
- Quay về 0: (15)

=> Biểu diễn: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

=> Tổng chi phí:

$$0 \rightarrow 1 = 10$$

$$1 \rightarrow 3 = 25$$

$$3 \rightarrow 2 = 30$$

$$2 \rightarrow 0 = 15$$

$$\Rightarrow \text{Tổng} = 80$$

Branch & Bound:

+ Thử tất cả đường đi hợp lệ

+ Cắt nhánh nếu: chi phí hiện tại > best

=> Cây trạng thái:

Start: 0

Try Level 1:

→ 1

→ 2

→ 3

Try Level 2:

0 → 1 → 2

0 → 1 → 3

0 → 2 → 1

0 → 2 → 3

0 → 3 → 1

0 → 3 → 2

...

=> Các đường đi:

Đường đi	Chi phí
0→1→2→3→0	$10+35+30+20 = 95$
0→1→3→2→0	$10+25+30+15 = 80$
0→2→1→3→0	$15+35+25+20 = 95$
0→2→3→1→0	$15+30+25+10 = 80$
0→3→1→2→0	$20+25+35+15 = 95$
0→3→2→1→0	$20+30+35+10 = 95$

=> Kết quả tối ưu: 0 → 1 → 3 → 2 → 0 hoặc 0 → 2 → 3 → 1 → 0

=> Output: 80

- Kết quả chạy code:

C: > University Coding Project > .vscode > el4_pttkgt.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n = 4;
5  int cost[4][4] = {
6      {0,10,15,20},
7      {10,0,35,25},
8      {15,35,0,30},
9      {20,25,30,0}
10 };
11
12 int visited[20], best = INT_MAX;
13
14 void Try(int u, int cnt, int sum){
15     if(cnt == n){
16         best = min(best, sum + cost[u][0]);
17         return;
18     }
19     for(int v=0;v<n;v++){
20         if(!visited[v]){
21             visited[v]=1;
22             Try(v, cnt+1, sum+cost[u][v]);
23             visited[v]=0;
24         }
25     }
26 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26200.8117]
(c) Microsoft Corporation. All rights reserved.

C:\University Coding Project\Giao hàng shipper - PTTKGT Project>c

c:\University Coding Project\.vscode\output>.\"el4_pttkgt.exe"

Ket qua: 80

c:\University Coding Project\.vscode\output>